

Code Optimizations

This document outlines several technical optimizations that can improve the performance, safety, and maintainability of the codebase.

1. O(1) Cart Lookups

Current: `Cart.java` uses a `List<CartItem>` to store items. Adding, updating, or removing items requires iterating over the list in $O(N)$ time.

Optimization: Change `items` to a `Map<String, CartItem>` or `LinkedHashMap<String, CartItem>` (to preserve insertion order) where the key is the `Product` ID. This reduces lookup, update, and removal time complexity to $O(1)$.

2. Memoization of Totals

Current: When `calculateTotal()` is invoked in `Cart.java`, it dynamically calculates the tax, discount, and subtotal. Since tax relies on discount and subtotal, and discount relies on subtotal, `calculateSubtotal()` is called redundantly multiple times in a single chain.

Optimization: Cache the `subtotal`, `discount`, and `tax` when the cart state is modified (e.g., in a `recalculateTotals()` private method). Alternatively, pass the subtotal down as an argument:

```
public double calculateTotal() {
    double subtotal = calculateSubtotal();
    double discount = calculateDiscount(subtotal);
    double tax = calculateTax(subtotal, discount);
    return Utils.roundToTwoDecimals(subtotal - discount + tax);
}
```

3. Explicit UTF-8 Encoding for File I/O

Current: `FileHandler.java` relies on the JVM's default Charset, which can break the serialization and deserialization of the Rupee symbol (₹) on Windows systems.

Optimization: Wrap IO classes with `StandardCharsets.UTF_8`:

```
new BufferedReader(new InputStreamReader(new FileInputStream(filePath), S
```

4. StringBuilder for CSV Writing

Current: `FileHandler.saveProducts()` constructs the output line using multiple `+` string concatenations in a loop.

Optimization: String concatenation in a loop generates intermediate String objects. While the compiler optimizes this slightly, explicitly using `String.format()` or `StringBuilder` or `String.join()` makes it more robust and slightly faster for large files.

5. Immutable Products & Deep Copies

Current: `Main.java` passes references to `Product` objects directly from the `catalog` Map into the `CartItem`. When `Cart.checkout()` reduces the stock, it mutates the catalog instances directly.

Optimization: While this works for a single-user CLI application, in a concurrent environment, this would cause race conditions. It is better to use immutable states where feasible or ensure that Cart objects use copies or just IDs, letting a centralized inventory service handle stock decrements atomically.